TECHNICAL SPECIFICATION & ARCHITECTURE BLUEPRINT

Automated Security Guard Scheduler (סידור אמריקאי) with flexible capacity rules

1. EXECUTIVE SUMMARY

This technical specification establishes the operational rules, database schemas, Row-Level Security (RLS) policies, and mathematical optimization models for a custom security guard scheduling application (אמריקאי סידור). The application allows active security guards to submit their weekly shift availabilities directly from their smartphones, eliminating manual coordination via WhatsApp. A backend optimization engine utilizing Google OR-Tools (CP-SAT Solver) then reads these preferences alongside manager-defined target quantities (כמות) and shift rules to compile a mathematically optimal, conflict-free roster in seconds. Both the web interface and database are hosted on fully free cloud tiers (Vercel and Supabase), keeping ongoing server and maintenance costs at exactly $0/month for your security team.

2. REVISED OPERATIONAL SCHEDULING RULES

To ensure safe, legal, and reliable facility coverage, the scheduling system enforces a strict hierarchy of capacities and floors:

A. Dynamic Roster Size

The scheduler is built around a dynamic roster that automatically scales based on the active guard list in the database (e.g., smoothly adapting between your 10 registered guards on paper and the 8 guards currently active). All constraints adapt to the size of the roster variable N.

• Active Guards:) יובלYuval) לוק ,(Luke) הודיה ,(Hodayah) שלי ,(Shelly) ים ,(Yam) עידן ,(Idan) ניתאי ,(Nitai) רום ,(Rom) אדיר ,(Adir) בן ,(Ben).

B. Daily Shift Structure

The working day is split into three distinct 8-hour blocks: Morning Shift (בוקר): 06:45 – 14:45, Afternoon Shift (צהריים): 14:45 – 22:45, and Night Shift (לילה): 22:45 – 06:45.

C. Operational Coverage: Targets vs. Critical Floors

To prevent under-staffing during guard fluctuations, the system defines separate Target Capacity (ideal) and Critical Floor (minimum safe) levels for each daily shift:

Shift Class	Target Capacity	Critical Operational Floor
Morning Shift (בוקר)	Changeable (2 or 3 Guards)	1 Guard (if target is 2), 2 Guards (if target is 3)
Afternoon Shift (צהריים)	Fixed at 2 slots	At least 1 Guard
Night Shift (לילה)	Fixed at 2 slots	At least 1 Guard

3. FLEXIBLE USER PREFERENCE & 12-HOUR SHIFT SCHEMA

A. Graceful Fallback for Empty Preferences

If a guard cannot work during a given week, they are permitted to leave their preference sheet entirely empty. The database handles this gracefully as a set of zero-weight preferences, and as long as their manager-defined target shifts (כמות) is set to 0 for that week, the solver will bypass scheduling them.

B. 12-Hour Shift Opt-In Modeling

To support guards who are willing to work extended 12-hour shifts, the preference form contains an opt-in toggle. 12h Day Shift Opt-In allows Morning + Afternoon shifts. 12h Night Shift Opt-In allows Afternoon + Night shifts. If a guard does not opt in, the solver strictly enforces the at-most-one-shift-per-day rule.

4. DATABASE & BACKEND ARCHITECTURE (SUPABASE VS. MONGODB)

Supabase (built on PostgreSQL) is selected over MongoDB as the core database BaaS. The table below outlines the architectural reasons for this decision:

Feature / Aspect	Supabase (PostgreSQL)	MongoDB (Atlas Document)
API Connectivity	Automatic PostgREST: Instantly exposes secure REST endpoints directly on your tables. No backend code required.	Retired Atlas Data API: Retired in Sept 2025. Today, you are forced to host a custom REST proxy server on Render/Railway.
Data Integrity	Relational: Strictly links Employees, Wish Requests, and Schedules via foreign keys.	Document-based: Flexible, but struggles with multi-table links and conflict auditing.
Row-Level Security (RLS)	Database Engine Enforced: Restricts access directly on the database engine. Zero-trust by design.	Application Layer: Rules reside above the database, risking exposure if API configurations are bypassed.

5. COMPLETE SUPABASE SQL SCHEMA

Copy-paste these commands directly into your Supabase SQL Editor to deploy your tables and configure your Row-Level Security rules:

```
-- 1. Profiles Table (with active flag for dynamic roster)
CREATE TABLE profiles (
  id UUID REFERENCES auth.users ON DELETE CASCADE PRIMARY KEY,
  display_name TEXT NOT NULL,
  role TEXT DEFAULT 'guard' CHECK (role IN ('admin', 'manager', 'guard')),
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT TIMEZONE('utc'::text, NOW()) NOT NULL
);

-- 2. Weekly Wish Preferences
CREATE TABLE weekly_wish_preferences (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  guard_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
  target_date DATE NOT NULL,
  shift_type TEXT CHECK (shift_type IN ('morning', 'afternoon', 'night')),
  is_available BOOLEAN DEFAULT TRUE,
  can_work_12h_day BOOLEAN DEFAULT FALSE,
  can_work_12h_night BOOLEAN DEFAULT FALSE,
  UNIQUE(guard_id, target_date, shift_type)
);

-- 3. RLS Rules Configuration
ALTER TABLE weekly_wish_preferences ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Guards can only manage their own preferences"
ON weekly_wish_preferences FOR ALL TO authenticated USING (auth.uid() = guard_id);
```

6. MATHEMATICAL SCHEDULING SOLVER MODEL (GOOGLE OR-Tools CP-SAT)

The optimization model utilizes CP-SAT to maximize preference satisfaction while strictly enforcing capacity target shifts and critical floors:

```
from ortools.sat.python import cp_model

def generate_optimal_schedule(active_guards, manager_settings):
    model = cp_model.CpModel()
    num_guards = len(active_guards)
    num_days = 7
    num_shifts = 3

    # Variables: x[g, d, s] is 1 if guard g works shift s on day d, 0 otherwise
    x = {}
    for g in range(num_guards):
        for d in range(num_days):
            for s in range(num_shifts):
                x[g, d, s] = model.NewBoolVar(f'x_g{g}_d{d}_s{s}')

    # Day Shift (0) Coverage constraint with Changeable Floors
    day_target = manager_settings['day_shift_target_slots']
    day_floor = 2 if day_target == 3 else 1
    for d in range(num_days):
        model.Add(sum(x[g, d, 0] for g in range(num_guards)) >= day_floor)
        model.Add(sum(x[g, d, 0] for g in range(num_guards)) <= day_target)

    # Rest Constraint (No-Clopen)
    for g in range(num_guards):
        for d in range(num_days - 1):
            model.Add(x[g, d, 2] + x[g, d+1, 0] <= 1)

    # Solve
```

```
solver = cp_model.CpSolver()
status = solver.Solve(model)
return status
```

7. AUTOMATED DEPLOYMENT & HOSTING PIPELINE

Set up your free hosting pipeline with GitHub and Vercel: 1. Push code to GitHub repository 'guard-scheduler'. 2. Connect Vercel to repository. 3. Input keys NEXT_PUBLIC_SUPABASE_URL and NEXT_PUBLIC_SUPABASE_ANON_KEY. 4. Share mobile-responsive URL (e.g. guard-scheduler.vercel.app) with guards.

8. OPERATIONS & ADMINISTRATION WORKFLOW

Implement the 'Draft and Audit' workflow. First, guards submit preferences by Thursday. Second, the manager defines 'כמות' and triggers the solver draft. Finally, the manager audits and approves before publishing live roster.